ENTERPRISE VULNERABILITY REMEDIATION PLATFORM

How the backend is built and how every important piece works

This deck explains the FastAPI backend, the database layer, the Wazuh API integration, the Ansible/AWX integration, the sync logic, the remediation workflow, and the testing strategy.

> Backend location: **`/home/ilyass/Desktop/PA_development/backend`**

01

The backend is the control center of the platform.

Detection

Wazuh API

The backend authenticates to Wazuh, reads agents, reads vulnerabilities, and stores the result locally.

Traceability

PostgreSQL

Assets, vulnerabilities, remediation jobs, and audit logs are kept in our own database.

Correction

Ansible/AWX

The backend creates approved remediation jobs, then launches AWX job templates when allowed.

The backend does not replace Wazuh. It exploits Wazuh detection data and adds governance, storage, approval, and remediation orchestration.

Each folder has one clear responsibility.

Folder	Role	Why it matters
`app/api/routes`	REST endpoints	Receives HTTP requests from Swagger, frontend, or curl.
`app/core`	Configuration and security	Loads environment variables and protects endpoints with API keys.
`app/db`	Database session	Creates SQLAlchemy engine and gives each request a database session.
`app/models`	Database tables	Defines assets, vulnerabilities, jobs, and audit logs.
`app/schemas`	API input/output shapes	Controls what JSON the API accepts and returns.
`app/services`	Business logic	Talks to Wazuh/AWX and performs sync/remediation workflows.

How one request moves through the backend.

1

Client

Frontend, Swagger, or curl sends an HTTP request.

2

FastAPI

The route validates parameters and JSON body.

3

Service

Business logic decides what should happen.

4

External API

Wazuh or AWX is called when needed.

5

Database

Result is stored and returned as JSON.

```
Client -> route -> dependency/security -> service -> Wazuh/AWX -> database -> JSON response
```

`app/main.py` creates and configures the API.

What it does

- Creates the FastAPI application.
- Registers CORS rules for the future frontend.
- Includes all API routes under **`/api/v1`**.
- Creates tables automatically in development.

```
app = FastAPI(
    title=settings.app_name,
    version="0.1.0",
    lifespan=lifespan,
)

app.include_router(
    api_router,
    prefix=settings.api_prefix,
    dependencies=[Depends(require_api_key)],
)
```

The root **`/health`** endpoint stays public. The real platform API is protected under **`/api/v1`**.

`app/core/config.py` centralizes environment settings.

```
class Settings(BaseSettings):
    app_name: str
    app_env: Literal["development", "test", "production"]
    database_url: str

    wazuh_api_url: str
    wazuh_api_user: str
    wazuh_api_password: str
    wazuh_verify_tls: bool

    awx_api_url: str
    awx_api_user: str | None
    awx_api_password: str | None
    awx_api_token: str | None
```

Why this is important

- Secrets are not hardcoded in Python files.
- Local, test, and production environments can use different values.
- Wazuh and AWX credentials are loaded from **`.env`**.
- Base URLs are normalized to avoid double slashes.

`app/core/security.py` protects the API with an API key.

How it works

- If **`APP_API_KEY`** is configured, every protected request must provide it.
- The key is sent in the **`X-API-Key`** header.
- Wrong or missing keys receive HTTP 401.
- This is a first security layer before adding SSO/RBAC later.

```
expected_key = get_settings().app_api_key
provided_key = request.headers.get("x-api-key")

if provided_key != expected_key:
    raise HTTPException(
        status_code=401,
        detail="Missing or invalid API key.",
    )
```

For enterprise production, this will evolve into SSO, JWT, RBAC, audit trails, and per-role permissions.

`app/db/session.py` gives each request safe database access.

```
engine = create_engine(settings.database_url, pool_pre_pi
SessionLocal = sessionmaker(
    bind=engine,
    autocommit=False,
    autoflush=False,
)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Key idea

FastAPI injects a database session into routes. The route uses it, then the session is closed automatically.

- No global database cursor.
- No forgotten open connection.
- Works with PostgreSQL and SQLite tests.

The ORM models define the platform memory.

Core tables

- **`Asset`**: every Wazuh agent becomes one enterprise asset.
- **`Vulnerability`**: each detected CVE is linked to an asset.
- **`RemediationJob`**: every correction request has a controlled lifecycle.
- **`AuditLog`**: important actions are recorded for traceability.

Shared mixins

The mixins add a UUID primary key and timestamps to all important tables.

```
id: str
created_at: datetime
updated_at: datetime
```

This database is our platform state. Wazuh is the detection source, but PostgreSQL is the platform history and workflow database.

Asset Model

`models/asset.py` stores machines discovered from Wazuh.

```
class Asset(Base, UUIDPrimaryKeyMixin, TimestampMixin):
    wazuh_agent_id: str
    hostname: str
    ip_address: str | None
    os_name: str | None
    os_version: str | None
    os_platform: str | None
    agent_version: str | None
    status: str
    last_seen: datetime | None
    raw_wazuh: dict | None
```

Why these fields matter

- **`wazuh_agent_id`** links our asset to Wazuh.
- **`hostname`** and **`ip_address`** identify the machine.
- **`os_platform`** helps choose Linux vs Windows remediation.
- **`raw_wazuh`** keeps the original payload for debugging.

`models/vulnerability.py` stores detected CVEs.

```
class Vulnerability(Base, UUIDPrimaryKeyMixin, TimestampMi
    asset_id: str
    cve: str
    title: str | None
    severity: str
    cvss_score: float | None
    package_name: str | None
    installed_version: str | None
    fixed_version: str | None
    status: str
```

Important design choice

A CVE is not stored alone. It is stored as a vulnerability affecting a specific asset and package.

- Same CVE can affect multiple machines.
- Same machine can have many CVEs.
- **`status`** moves from **`open`** to **`fixed`** after verification.

Correction must be controlled, approved, and traceable.

RemediationJob

- Links a correction to an asset and optional vulnerability.
- Stores the engine: currently **`ansible_awx`**.
- Stores approval fields and AWX job IDs.
- Stores result and error payloads for investigation.

AuditLog

- Records sync operations.
- Records remediation creation, approval, and launch.
- Stores actor, action, entity type, entity ID, and metadata.
- Creates accountability for enterprise use.

```
pending_approval -> approved -> running -> verification_pending -> verified_fixed
```

`app/schemas` defines the JSON contract.

Why schemas are separate from models

- Models describe database tables.
- Schemas describe API request and response JSON.
- This prevents exposing internal database details accidentally.
- FastAPI uses schemas to generate Swagger documentation.

```
class RemediationJobCreate(BaseModel):
    asset_id: str
    vulnerability_id: str | None = None
    engine: Literal["ansible_awx", "manual"]
    action: Literal["package_update", ...]
    created_by: str
    awx_job_template_id: int | None
    requested_payload: dict[str, Any]
```

`services/normalizers.py` converts Wazuh JSON into our format.

```
def normalize_agent(raw):
    os_info = raw.get("os") or {}
    return {
        "wazuh_agent_id": str(raw.get("id")),
        "hostname": raw.get("name"),
        "ip_address": raw.get("ip"),
        "os_platform": os_info.get("platform"),
        "agent_version": raw.get("version"),
        "status": raw.get("status"),
        "raw_wazuh": raw,
    }
```

Problem solved

External APIs do not always return exactly the shape we want. Normalizers create stable internal records.

- Handles nested Wazuh fields.
- Parses dates safely.
- Preserves the original raw data.
- Makes tests easy and reliable.

Wazuh API Client

`services/wazuh_client.py` handles Wazuh authentication and requests.

Responsibilities

- Authenticate using Wazuh user and password.
- Store the JWT token for future requests.
- Retry authentication if Wazuh returns 401.
- Convert network/API failures into clean backend errors.

```
POST /security/user/authenticate
Authorization: Basic user:password

GET /agents
Authorization: Bearer token

GET /vulnerability/{agent_id}
Authorization: Bearer token
```

The backend never asks the frontend to talk directly to Wazuh. It hides Wazuh credentials and exposes controlled platform endpoints.

The client is designed for many agents and many vulnerabilities.

```
params = {"limit": 500, "offset": 0}

while True:
    payload = await self.request("GET", "/agents", params=
    items = extract_wazuh_items(payload)
    agents.extend(items)

    if len(agents) >= total:
        break
    params["offset"] += limit
```

Why this matters

- Small labs may have 3 agents, but enterprises may have thousands.
- Pagination prevents missing results after the first page.
- The same pattern is used for vulnerabilities per agent.

Sync Agents

`WazuhSyncService.sync_agents` imports machines into PostgreSQL.

Wazuh API
GET /agents

->

PostgreSQL
assets table

Logic

- Fetch agents from Wazuh.
- Normalize each agent payload.
- If the asset exists, update it.
- If it is new, create it.

```
existing = db.execute(
    select(Asset).where(
        Asset.wazuh_agent_id == normalized["wazuh_agent_i
    )
).scalar_one_or_none()
```

`sync_vulnerabilities` imports CVEs for every asset.

Main algorithm

- Read all assets from our database.
- For each asset, call Wazuh vulnerability endpoint.
- Normalize each CVE record.
- Create or update vulnerability rows.
- Mark absent old CVEs as fixed.

```
seen_keys.add((cve, package_name))

if vulnerability:
    vulnerability.status = "open"
else:
    db.add(Vulnerability(...))

mark_absent_vulnerabilities_fixed(...)
```

The verification idea is already here: if Wazuh no longer reports a previously open CVE, the backend can mark it as fixed.

`services/ansible_client.py` calls Ansible Automation Controller.

```
async def launch_job_template(
    self,
    job_template_id: int,
    extra_vars: dict[str, Any],
):
    return await self.request(
        "POST",
        f"/api/v2/job_templates/{job_template_id}/launch/
        json={"extra_vars": extra_vars},
    )
```

How we use AWX

- AWX stores approved playbooks and job templates.
- The backend does not execute shell commands directly.
- The backend sends controlled variables to AWX.
- AWX runs the remediation on target machines.

Remediation Workflow

`services/remediation_service.py` controls the correction lifecycle.

1

Create

User creates a remediation job for an asset/CVE.

2

Approve

An authorized person approves the action.

3

Launch

Backend launches the AWX job template.

4

Refresh

Backend checks AWX execution status.

5

Verify

Wazuh rescan confirms whether the CVE disappeared.

```
pending_approval -> approved -> running -> verification_pending -> verified_fixed
```

AWX Extra Vars

The backend sends AWX the exact context needed for remediation.

```
extra_vars = {
    "pa_job_id": job.id,
    "asset_hostname": asset.hostname,
    "asset_ip_address": asset.ip_address,
    "asset_os_platform": asset.os_platform,
    "cve": vulnerability.cve,
    "package_name": vulnerability.package_name,
    "installed_version": vulnerability.installed_version,
    "fixed_version": vulnerability.fixed_version,
    "remediation_action": job.action,
}
```

Why this is useful

- Playbooks know which asset to fix.
- Playbooks know which package/CVE is involved.
- AWX execution can be linked back to our job ID.
- The same pattern works for many remediation types.

The routes expose the platform functionality.

Endpoint group	Purpose	Examples
`/health`	Backend and database health	Check that API and DB are alive.
`/integrations`	External API health	Check Wazuh and AWX reachability.
`/sync`	Import Wazuh data	Sync agents and vulnerabilities.
`/assets`	Machine inventory	List assets and asset vulnerabilities.
`/vulnerabilities`	CVE view	Filter by asset, severity, status, CVE.
`/remediation/jobs`	Correction workflow	Create, approve, launch, refresh jobs.
`/audit-logs`	Traceability	Show sync and remediation history.

The backend is made to be reproducible.

Alembic

Keeps database schema changes versioned. First migration creates assets, vulnerabilities, jobs, and audit logs.

Docker Compose

Starts PostgreSQL and Redis locally, giving us a consistent development environment.

Pytest

Tests health endpoints and Wazuh normalizers. It uses SQLite so tests do not depend on PostgreSQL.

```
docker compose up -d postgres redis
alembic upgrade head
uvicorn app.main:app --reload --port 8000
pytest -v
```

The backend foundation is ready for the frontend and real AWX playbooks.

Ready now

- FastAPI backend structure.
- PostgreSQL schema and migration.
- Wazuh API client and sync logic.
- AWX API client and remediation workflow.
- Audit logs and testing setup.

Next development steps

- Add real user authentication and RBAC.
- Create AWX job templates and playbooks.
- Add verification endpoint after remediation.
- Build frontend pages for assets, CVEs, jobs, and approvals.
- Add production deployment security hardening.

The most important end-to-end test remains: Wazuh health -> sync agents -> sync vulnerabilities -> create job -> approve -> launch in AWX -> resync and verify.